

Reporte de Práctica 3: Instalación de Entorno Haskell y Aplicación TODO

Introducción

Haskell es un lenguaje de programación funcional que se caracteriza por el uso de funciones puras, la inmutabilidad de los datos y una sintaxis diferente a lenguajes imperativos como C. A diferencia de estos Haskell no se basa en la modificación de variables sino en la evaluación de expresiones.

El objetivo de esta práctica es instalar el entorno de desarrollo de Haskell, familiarizarse con sus herramientas principales y ejecutar una aplicación de ejemplo tipo TODO para comprender su funcionamiento básico.

Instalación del entorno de desarrollo

Para comenzar se accedió a la página oficial de descargas de Haskell donde se copió el comando proporcionado en dicha página y se ejecutó en una ventana de PowerShell sin privilegios de administrador.

Durante este proceso se instalaron varias herramientas importantes:

- **GHCCup:** gestor que facilita la instalación del entorno de Haskell.
- **GHC** (Glasgow Haskell Compiler): compilador principal del lenguaje.
- **Hugs:** intérprete interactivo de Haskell.
- **HLS (Haskell Language Server):** proporciona soporte de lenguaje y librerías estándar.
- **Stack:** manejador de paquetes y dependencias.
- **Cabal:** herramienta de construcción que trabaja junto con Stack y GHC.

Los archivos de Haskell utilizan la extensión .hs lo que permite identificar fácilmente el código fuente. Una vez finalizada la instalación se verificó el correcto funcionamiento ejecutando comandos en la terminal confirmando que las herramientas estaban disponibles.

Introducción al lenguaje Haskell

Previo a la ejecución de la aplicación se revisaron recursos de introducción a Haskell. Esto permitió comprender las diferencias principales entre paradigmas destacando que Haskell utiliza funciones puras, evita el uso de variables mutables y se basa en la evaluación de expresiones. Aunque no se profundizó completamente en el lenguaje se logró entender su sintaxis básica y la manera en que se estructuran los programas.

Descarga y preparación del proyecto

Para trabajar con la aplicación TODO se utilizó un repositorio público. Primero se clonó el repositorio completo utilizando Git mediante el comando:

```
git clone https://github.com/steadylearner/Haskell.git
```

Posteriormente se accedió a la carpeta específica del proyecto:

```
cd Haskell/examples/blog/todo
```

Esto permitió ubicarse en el directorio donde se encuentra el código fuente de la aplicación.

Compilación del proyecto

Una vez dentro de la carpeta del proyecto se procedió a compilar la aplicación utilizando la herramienta Stack con el siguiente comando:

```
stack build
```

Durante este proceso Stack descargó las dependencias necesarias incluyendo el compilador y las librerías requeridas. Este paso tomó varios minutos debido al tamaño de los archivos descargados lo cual es normal en la primera ejecución.

Ejecución de la aplicación

Después de compilar correctamente el proyecto se ejecutó la aplicación mediante el comando:

```
stack exec todo-exe
```

Al ejecutarse la aplicación mostró un menú de comandos en la terminal indicando que estaba lista para interactuar con el usuario. Inicialmente intentó abrir una dirección en `http://localhost:3000` sin embargo el navegador mostró un error debido a que no existe un servidor activo en esa dirección.

Durante la ejecución también se presentó un error relacionado con la ausencia del archivo `.env` el cual fue solucionado creando dicho archivo manualmente e incluyendo la variable requerida:

```
WEBSITE=http://localhost:3000
```

Funcionamiento de la aplicación TODO

La aplicación funciona desde la línea de comandos y permite gestionar una lista de tareas. El usuario puede interactuar mediante diferentes comandos:

- Agregar tareas
- Listar tareas
- Eliminar tareas
- Editar tareas
- Limpiar la lista

Por ejemplo al ingresar el comando para agregar una tarea esta se almacena en una lista interna. Posteriormente se puede visualizar con el comando de listado o eliminar mediante su índice.

Este tipo de aplicación demuestra cómo el paradigma funcional puede utilizarse para desarrollar programas interactivos aunque con un enfoque diferente al de lenguajes tradicionales como C.

Conclusión

En esta práctica se logró instalar correctamente el entorno de desarrollo de Haskell y ejecutar una aplicación funcional tipo TODO. A pesar de que el proceso de instalación puede ser pesado debido a la descarga de múltiples componentes una vez configurado permite trabajar de manera eficiente con el lenguaje.

El uso de Haskell resultó interesante ya que presenta un enfoque distinto al de lenguajes imperativos. La ausencia de variables mutables y el uso de funciones puras cambian la forma de pensar al programar lo cual puede resultar complicado al inicio pero también ofrece ventajas en cuanto a claridad y organización del código.

En lo personal esta práctica permitió comprender de manera básica el paradigma funcional y observar su aplicación en un programa real. Aunque Haskell puede parecer complejo al principio demuestra ser un lenguaje poderoso y útil para el desarrollo de software estructurado.

Enlaces del proyecto

Página estática del reporte

El reporte completo puede consultarse en la siguiente página:

[Ver página del proyecto](#)

Repositorio de GitHub

El código fuente del proyecto se encuentra disponible en el siguiente repositorio:

[Ver repositorio en GitHub](#)

8. Referencias

Steadylearner. (s.f.). Todo example (Haskell blog example). GitHub.
<https://github.com/steadylearner/Haskell/tree/main/examples/blog/todo>

HaskellWiki. (s.f.). Haskell tutorial for C programmers. https://wiki.haskell.org/index.php?title=Haskell_Tutorial_for_C_Programmers

Steadylearner. (s.f.). How to use Stack to build a Haskell app. DEV Community.
<https://dev.to/steadylearner/how-to-use-stack-to-build-a-haskell-app-499j>